Client-Mediated Token: A Practical Approach to Secure QUIC Server-Side Migration

1. Problem Statement

In QUIC server-side migration (RFC 9000 Section 9.6), the primary server advertises a preferred address during the TLS handshake. After migration, the preferred server must possess the TLS session keys to decrypt client packets. Currently, no standard defines how the preferred server obtains these keys.

Our implementation transfers 445 bytes of migration state (including raw TLS secrets) over a plaintext TCP connection between the primary and preferred servers. This creates a critical attack surface: anyone who can observe or intercept this transfer obtains full session keys.

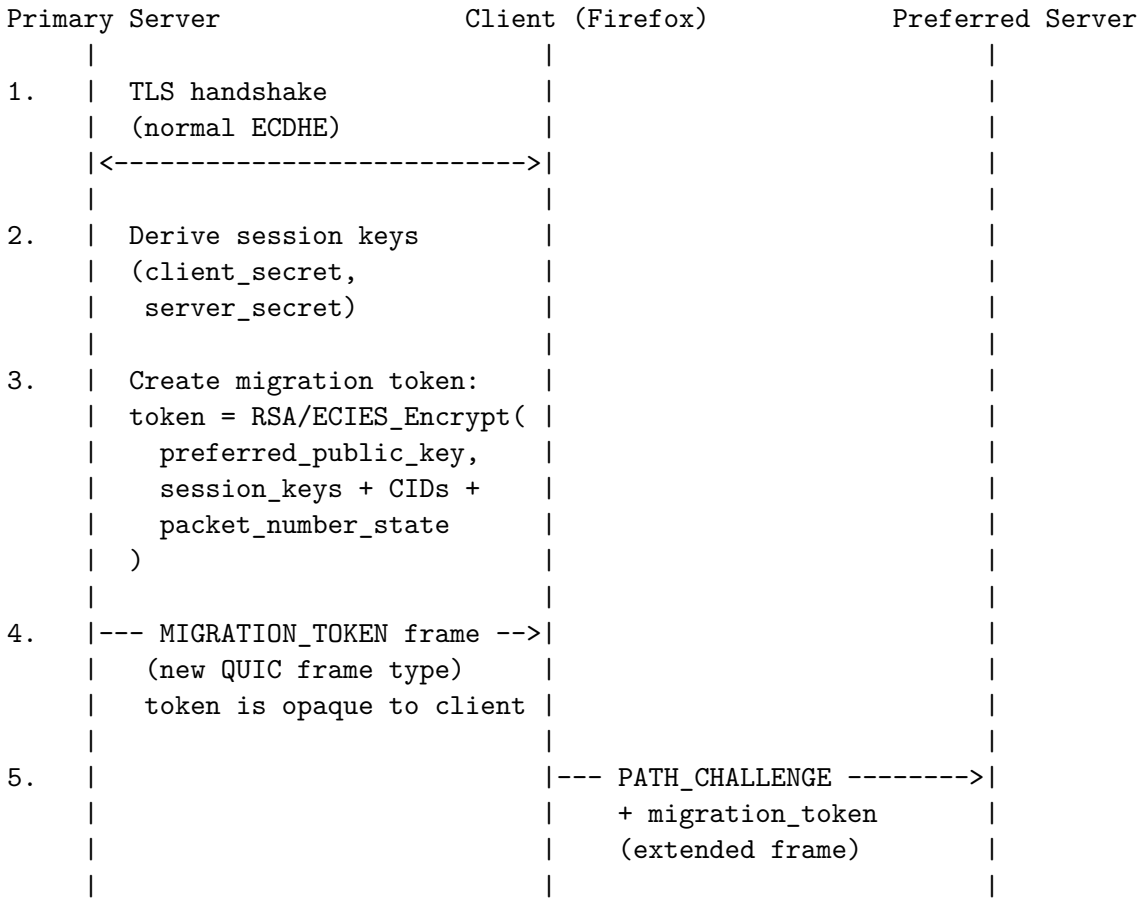
Goal: Eliminate the server-to-server secret transfer channel entirely.

---

2. Proposed Solution: Client-Mediated Token

2.1 Core Idea

Instead of the primary server sending secrets directly to the preferred server, the **client** carries an encrypted token from the primary to the preferred. The token contains the session keys, encrypted with the preferred server's public key. Only the preferred server can decrypt it.



```
6.   |                              |    Preferred decrypts:     |
     |                              |    keys = RSA/ECIES_Dec(   |
     |                              |      preferred_private_key,|
     |                              |      token                 |
     |                              |    )                       |
     |                              |                            |
7.   |                              |<-- PATH_RESPONSE ----------|
     |                              |    (encrypted with         |
     |                              |     derived keys)          |
     |                              |                            |
8.   |                              |    Migration complete.     |
     |                              |    Client talks to         |
     |                              |    preferred server now.   |
```

2.2 What Changes in the QUIC Protocol

New Transport Parameter: preferred_server_key Added to the TLS handshake alongside `preferred_address`:

```
preferred_server_key (0xTBD):
    key_type:       u8        (0x01 = RSA, 0x02 = ECDSA/P-256, 0x03 = Ed25519)
    public_key:     <var>     (DER-encoded public key of the preferred server)
    cert_hash:      [32]u8    (SHA-256 hash of preferred server's certificate)
```

This tells the client: "The preferred server's public key is X. Use it to encrypt the migration token."

New Frame: MIGRATION_TOKEN (Primary –> Client)

```
MIGRATION_TOKEN frame (type 0xTBD):
    token_length:   varint    (length of encrypted token)
    encrypted_token: [var]u8  (opaque to client -- cannot decrypt)
```

Sent by the primary server to the client after the handshake completes. The client stores it but cannot read it.

Extended PATH_CHALLENGE (Client –> Preferred)

```
PATH_CHALLENGE_WITH_TOKEN (type 0xTBD):
    challenge_data:  [8]u8    (standard 8-byte challenge)
    token_length:    varint   (length of migration token)
    migration_token: [var]u8  (forwarded from MIGRATION_TOKEN frame)
```

The client sends this instead of a standard PATH_CHALLENGE when migrating to a preferred address that included a `preferred_server_key`.

2.3 Token Structure (Inside the Encryption)

Before encryption, the migration token plaintext contains:

```
Migration Token Plaintext:
    magic:              "MTOK" (4 bytes)
    version:            u32 (QUIC version)
```

```
    write_secret:       [32]u8 (server -> client TLS secret)
    read_secret:        [32]u8 (client -> server TLS secret)
    next_write_secret:  [32]u8 (for key updates)
    next_read_secret:   [32]u8 (for key updates)
    cipher_suite:       u16 (TLS cipher identifier)
    epoch:              u32 (key generation number)
    used_pn_start:      u64
    used_pn_end:        u64
    min_pn:             u64
    local_cid_count:    u8
    local_cids:         [var] (connection IDs)
    remote_cid_count:   u8
    remote_cids:        [var] (connection IDs)
    client_addr:        [var] (IP + port)
    timestamp:          u64 (creation time, for expiry)
    nonce:              [16]u8 (random, prevents replay)

Total: ~500 bytes plaintext
Encrypted: ~550-800 bytes (depending on key type)
```

2.4 Encryption Options

Algorithm	Token Size	Speed	Security Level
RSA-OAEP (2048-bit)	256 bytes ciphertext per 190 bytes plaintext = ~768B total	Fast encrypt, slow decrypt	112-bit
ECIES (P-256)	~600 bytes (EC point + AES-GCM ciphertext)	Fast both ways	128-bit
ECIES (X25519)	~580 bytes (32B key + AES-GCM ciphertext)	Fastest	128-bit
HPKE (X25519 + AES-128-GCM)	~580 bytes	Fastest, modern standard	128-bit

Recommended: HPKE (RFC 9180) – already standardized, used in TLS Encrypted Client Hello (ECH), efficient, modern.

```
Encryption:
    (enc, ciphertext) = HPKE.Seal(preferred_public_key, token_plaintext)
    migration_token = enc || ciphertext

Decryption (on preferred server):
    token_plaintext = HPKE.Open(preferred_private_key, enc, ciphertext)
```

---

3. Security Analysis

3.1 What This Approach Protects Against

Threat	Protected?	How
Network sniffing between servers	Yes	No server-to-server connection exists
Unauthorized server receiving secrets	Yes	Only preferred server has the private key
Client reading the secrets	Yes	Token encrypted with preferred's key, not client's
Compromised Redis/middleman	Yes	No middleman exists
Man-in-the-middle on state transfer	Yes	No state transfer channel to intercept
Replay attack	Yes	Nonce + timestamp in token

3.2 What This Approach Does NOT Protect Against

Threat	Why Not
Malicious primary server sharing keys out-of-band	Primary has the keys from the handshake – it can always copy them
Compromised preferred server's private key	If attacker has the private key, they can decrypt any token
Primary redirecting to wrong preferred server	Client trusts whatever public key the primary provides
Client refusing to migrate	Client can simply not send PATH_CHALLENGE

3.3 QUIC-Exfil: Is It Prevented?

Partially, but not fully.

```
What IS prevented:
    Passive network attacker can no longer intercept migration state.
    No server-to-server channel exists to tap.

What is NOT prevented:
    Primary server still has the session keys (it did the TLS handshake).
    A malicious primary can:
      1. Create a token encrypted for an attacker's public key
      2. Advertise the attacker's address as preferred_address
      3. Client dutifully carries the token to the attacker
      4. Attacker decrypts the token with their private key
      5. QUIC-Exfil successful

    The client cannot distinguish a legitimate preferred server
```

```
    from an attacker's server -- it trusts whatever the primary says.
```

3.4 How to Mitigate the Remaining Risk

Certificate Transparency for Preferred Servers Require that the preferred server's certificate is logged in a public Certificate Transparency (CT) log, just like web certificates. The client can verify:

```
Client checks:
    1. preferred_server_key has a valid certificate (not just a raw key)
    2. The certificate is logged in a CT log
    3. The certificate is issued by a trusted CA
    4. The certificate covers the preferred_address domain/IP

If any check fails --> refuse to migrate
```

This raises the bar for QUIC-Exfil: the attacker needs a **publicly logged certificate** for their server, which is traceable and attributable.

Shared Certificate Authority Constraint Require that primary and preferred servers' certificates are issued by the **same CA** or are part of the same organizational certificate chain:

```
Client checks:
    primary_cert.issuer == preferred_cert.issuer

    OR

    primary_cert.organization == preferred_cert.organization
```

This prevents a malicious primary from redirecting to a completely unrelated server.

---

4. Implementation Sketch

4.1 Primary Server Changes

```rust
// After TLS handshake completes:
fn create_migration_token(
    session_keys: &SessionKeys,
    preferred_public_key: &HpkePublicKey,
) -> Vec<u8> {
    // 1. Serialize migration state
    let plaintext = MigrationToken {
        magic: *b"MTOK",
        write_secret: session_keys.write_secret_bytes(),
        read_secret: session_keys.read_secret_bytes(),
        // ... other fields ...
        nonce: random::<[u8; 16]>(),
        timestamp: SystemTime::now(),
```

```
    }.encode();

    // 2. Encrypt with preferred server's public key (HPKE)
    let (enc, ciphertext) = hpke::seal(
        preferred_public_key,
        b"quic-migration-token",  // info/context string
        &plaintext,
    );

    // 3. Wipe plaintext immediately
    secure_wipe(&mut plaintext);

    // 4. Return enc || ciphertext
    [enc, ciphertext].concat()
}

// Send to client via new QUIC frame
fn send_migration_token(conn: &mut Connection, token: &[u8]) {
    conn.send_frame(Frame::MigrationToken {
        token: token.to_vec(),
    });
}
```

4.2 Client Changes (Firefox/Browser)

```
// On receiving MIGRATION_TOKEN frame:
fn handle_migration_token(token: Vec<u8>) {
    // Store the token -- client CANNOT decrypt it
    self.migration_token = Some(token);
}

// When sending PATH_CHALLENGE to preferred address:
fn send_path_challenge_with_token(preferred_addr: SocketAddr) {
    let challenge_data = random::<[u8; 8]>();

    if let Some(token) = &self.migration_token {
        // Send extended PATH_CHALLENGE with token
        self.send_frame(Frame::PathChallengeWithToken {
            data: challenge_data,
            token: token.clone(),
        });
    } else {
        // Fallback: standard PATH_CHALLENGE (same-machine migration)
        self.send_frame(Frame::PathChallenge {
            data: challenge_data,
        });
```

```
    }
}
```

4.3 Preferred Server Changes

```
// On receiving PATH_CHALLENGE_WITH_TOKEN:
fn handle_path_challenge_with_token(
    challenge_data: [u8; 8],
    encrypted_token: &[u8],
    private_key: &HpkePrivateKey,
) -> Result<ConnectionState> {
    // 1. Decrypt the token with our private key
    let plaintext = hpke::open(
        private_key,
        b"quic-migration-token",
        encrypted_token,
    )?;

    // 2. Parse the migration state
    let token = MigrationToken::decode(&plaintext)?;

    // 3. Validate timestamp (reject expired tokens)
    if token.timestamp + MAX_TOKEN_AGE < SystemTime::now() {
        return Err("Token expired");
    }

    // 4. Import crypto keys (same as current implementation)
    let conn = import_state(&token)?;

    // 5. Wipe plaintext
    secure_wipe(&mut plaintext);

    // 6. Send PATH_RESPONSE
    send_path_response(&challenge_data, &conn);

    Ok(conn)
}
```

---

5. Comparison with Current Approach

Aspect	Current (TCP Push)	Client-Mediated Token
Server-to-server connection	Yes (TCP :9999)	None
Secrets on wire	Plaintext	Encrypted (HPKE)
Who carries the state	Direct TCP	Client (opaque token)

Aspect	Current (TCP Push)	Client-Mediated Token
Client can read secrets	N/A	No (encrypted for preferred)
Network attacker can intercept	Yes	No
Requires preferred server's public key	No	Yes (in handshake)
Latency	~660us (TCP)	~0 (piggybacked on PATH_CHALLENGE)
Complexity	Low	Medium
Requires client changes	No	Yes (new frame types)
Requires RFC changes	No	Yes (new extension)
Prevents passive QUIC-Exfil	No	Yes
Prevents active QUIC-Exfil	No	No (primary controls the public key)

---

6. Open Questions for Future Research

1. **Token size vs. PATH_CHALLENGE limits:** RFC 9000 Section 8.2.1 requires path validation packets to be at least 1200 bytes. The ~580-byte token fits within this, but should it be a separate frame or part of PATH_CHALLENGE?
2. **Token expiry:** How long should a migration token be valid? Too short and the client may not migrate in time. Too long and a captured token could be replayed.
3. **Multiple preferred servers:** If the RFC were extended to allow multiple preferred addresses, should each have its own public key? This would require multiple tokens.
4. **Backward compatibility:** How should a client behave if the primary sends a MIGRATION_TOKEN but the preferred server doesn't understand the extended PATH_CHALLENGE? Fallback to standard PATH_CHALLENGE (which would fail without state transfer).
5. **Key distribution:** How does the primary server obtain the preferred server's public key? Options: shared configuration, DNS (like HTTPS RR), or a new discovery protocol.
6. **Certificate pinning:** Should the client pin the preferred server's certificate after seeing it in the handshake? This would prevent the primary from changing the preferred server mid-connection.

---

7. Conclusion

The client-mediated token approach is the most practical path toward securing QUIC server-side migration without requiring fundamental changes to TLS cryptography. It eliminates the server-to-server secret transfer channel, uses standard public-key encryption (HPKE/RFC 9180), and integrates naturally with existing QUIC path validation.

However, it cannot fully prevent QUIC-Exfil because the primary server – which performs the TLS handshake – inherently possesses the session keys and can always share them through channels

outside the protocol's control. Addressing this would require either a true 3-party key exchange (fundamentally changing TLS) or external trust mechanisms like Certificate Transparency.

The approach represents a significant security improvement over the current state of the art (plaintext inter-server transfer) while remaining implementable within the existing QUIC/TLS framework with modest protocol extensions.